Object Recognition using CNN

DS - 303 Project

Satwik Murarka (190020101)
Arem Praveen Prakash (190020019)
Mihir Nandawat (190020071)

Honour Code

We declare that the contents of this project are not copied to the best of our knowledge and we have not indulged in any academic malpractice

Satwik Murarka (190020101)
Arem Praveen Prakash (190020019)
Mihir Nandawat (190020071)

Overview & Problem Definition

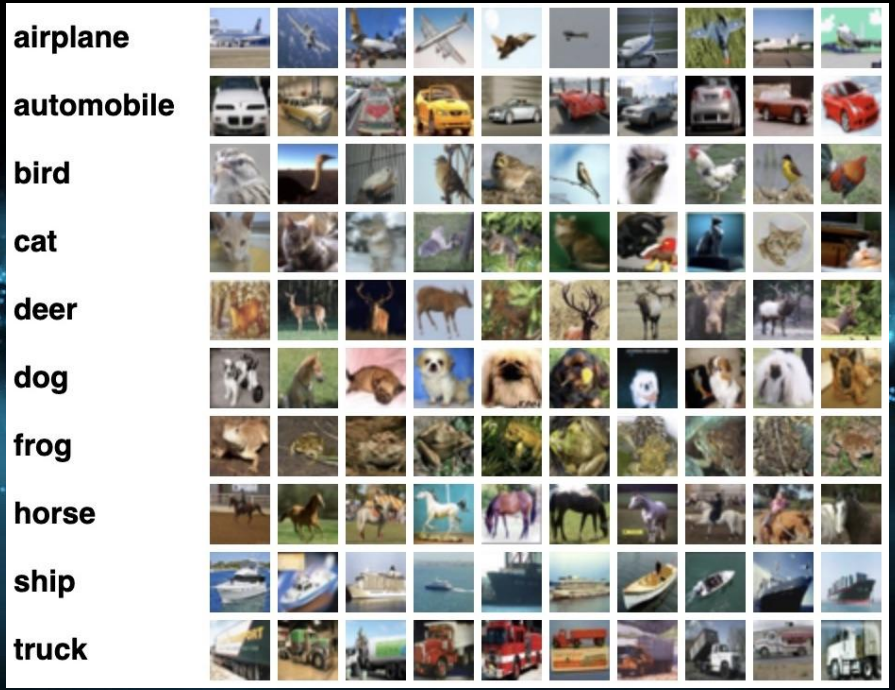
Object Detection is an important aspect of Computer Vision. It has a lot of important applications in real life ranging from autonomous vehicles, aerial surveillance to snapchat filters.

Given a set of images with multiple classes build and train an algorithm to solve the multi-class classification problem with decent accuracy.

Data

We used **CIFAR-10** dataset which consists of 50k labelled images with 10 classes such as airplane, bird etc. along with 10k unlabelled images.

The colored images are of size 32*32 pixels.



Solution

Traditional ANNs cannot be implemented since vectorization of pixels leads to loss of important spatial information of their arrangement. It would have excess of weights and biases which would make it computationally intensive.

So, we used CNN to solve this issue. It is better in the following aspects :-

- UnSupervised Feature Detection
- Pooling which downscales the image with minimal information loss
- Supports parallelization

All of these advantages makes it an excellent choice for this problem.

Method with Mathematical Details

Feature Extraction using Convolution

Filter is a set of weights which are used to extract local features. We use multiple filters to extract different features from an image.

Example: We use Vertical Edge filter (VEF) and Horizontal Edge Filter (HEF) to get the edges of an object. Where, VEF and HEF are matrices as shown in the below image.

$$VEF = \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix} = {}^{T}HEF$$

Convolution

Let the dimensions of the image I be (n_H , n_W , n_C) where, n_H is the size of the height, n_W is the size of width, n_C is the size of the channels, which is 3 for a RGB image.

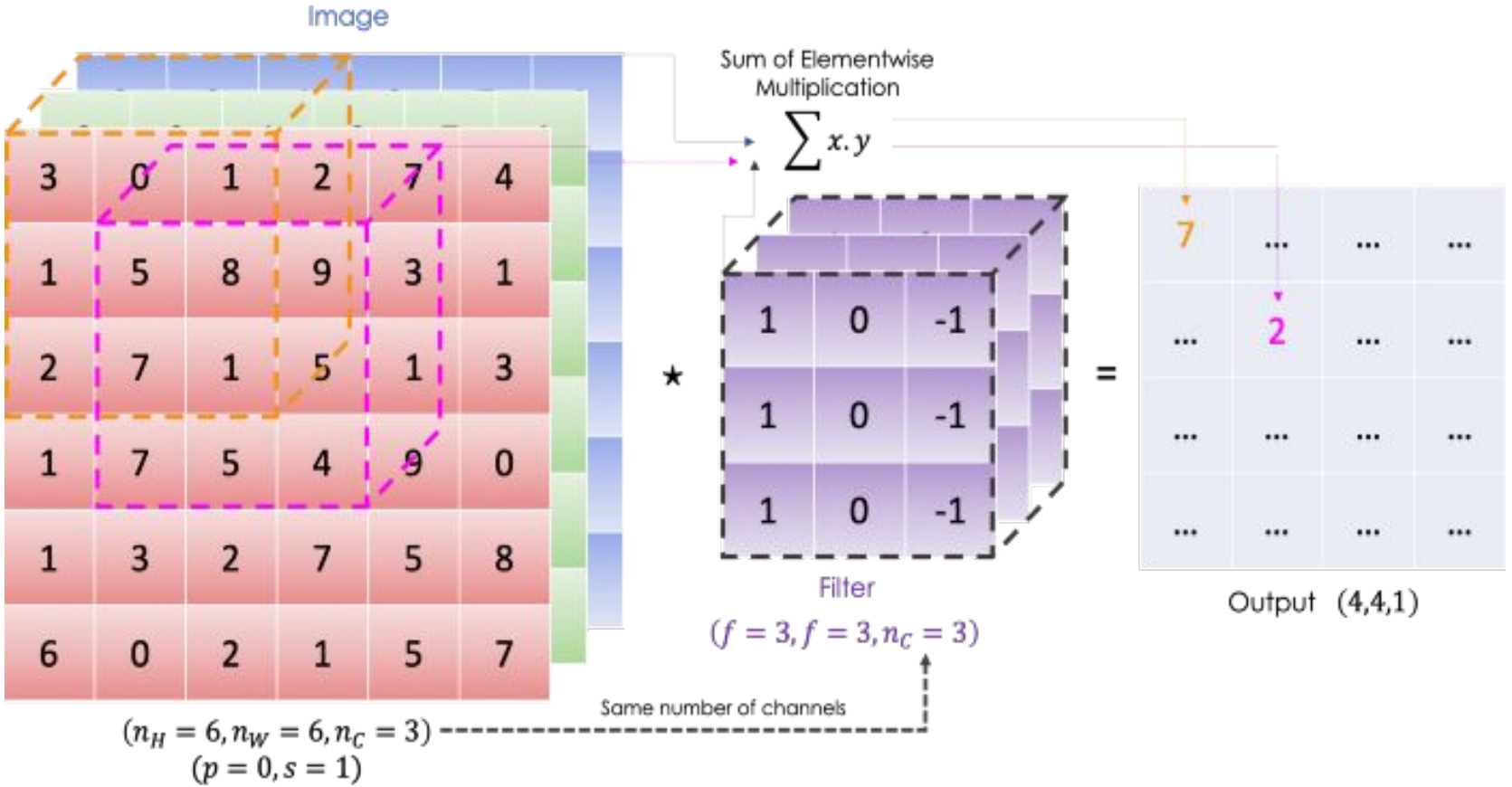
Let the dimensions of the filter K be (f, f , n_C) where, f is the dimension of filter K and it is considered to have an odd dimension, n_C is the size of the channels.

The convolutional product between the image and the filter is a 2D Matrix where each element of the output matrix is the sum of the element wise multiplication of the cube (filter) and the sub cube of the given image. The top and bottom images shown below are the mathematical equations for convolutional product and dimensions of the output matrix respectively

$$conv(I,K)_{x,y} = \sum_{i=1}^{n_H} \sum_{j=1}^{n_W} \sum_{k=1}^{n_C} K_{i,j,k} I_{x+i-1,y+j-1,k}$$

$$\begin{aligned} dim(conv(I,K)) &= \left(\left\lfloor \frac{n_H + 2p - f}{s} + 1 \right\rfloor , \left\lfloor \frac{n_W + 2p - f}{s} + 1 \right\rfloor \right); s > 0 \\ &= (n_H + 2p - f, n_W + 2p - f); s = 0 \end{aligned}$$

The below image shows an illustration of Convolution Product of an Image and Filter



Convolutional Layer

We apply convolutional products on the input using many filters, followed by an activation function (represented by ψ). In this project we used the Rectified Linear Unit (ReLU) as the Activation function. The Mathematical equations for the convolutional product and other things at l^{th} layer are shown in the image here

More preciously, at the $\mathbf{l^{th}layer}$, we denote:

- `Input` : $a^{[l-1]}$ with size $(n_H^{[l-1]}, n_W^{[l-1]}, n_C^{[l-1]})$, $a^{[0]}$ being the image in the input
- `Padding` : $p^{[l]}$, `stride` : $s^{[l]}$
- `Number of filters` : $n_C^{[l]}$ where each $K^{(n)}$ has the dimension: $(f^{[l]}, f^{[l]}, n_C^{[l-1]})$
- `Bias` of the nth convolution: $b_n^{[l]}$
- `Activation function` : $\psi^{[l]}$
- `Output` : $a^{[l]}$ with size $(n_H^{[l]}, n_W^{[l]}, n_C^{[l]})$

And we have:

$$\forall n \in [1, 2, ..., n_C^{[l]}] :$$

$$conv(a^{[l-1]}, K^{(n)})_{x,y} = \psi^{[l]}(\sum_{i=1}^{n_H^{[l-1]}} \sum_{j=1}^{n_W^{[l-1]}} \sum_{k=1}^{n_C^{[l-1]}} K_{i,j,k}^{(n)} a_{x+i-1,y+j-1,k}^{[l-1]} + b_n^{[l]})$$
$$dim(conv(a^{[l-1]}, K^{(n)})) = (n_H^{[l]}, n_W^{[l]})$$

Thus:

$$a^{[l]} =$$
$$[\psi^{[l]}(conv(a^{[l-1]}, K^{(1)})), \psi^{[l]}(conv(a^{[l-1]}, K^{(2)})), ..., \psi^{[l]}(conv(a^{[l-1]}, K^{(n_C^{[l]})}))]$$
$$dim(a^{[l]}) = (n_H^{[l]}, n_W^{[l]}, n_C^{[l]})$$

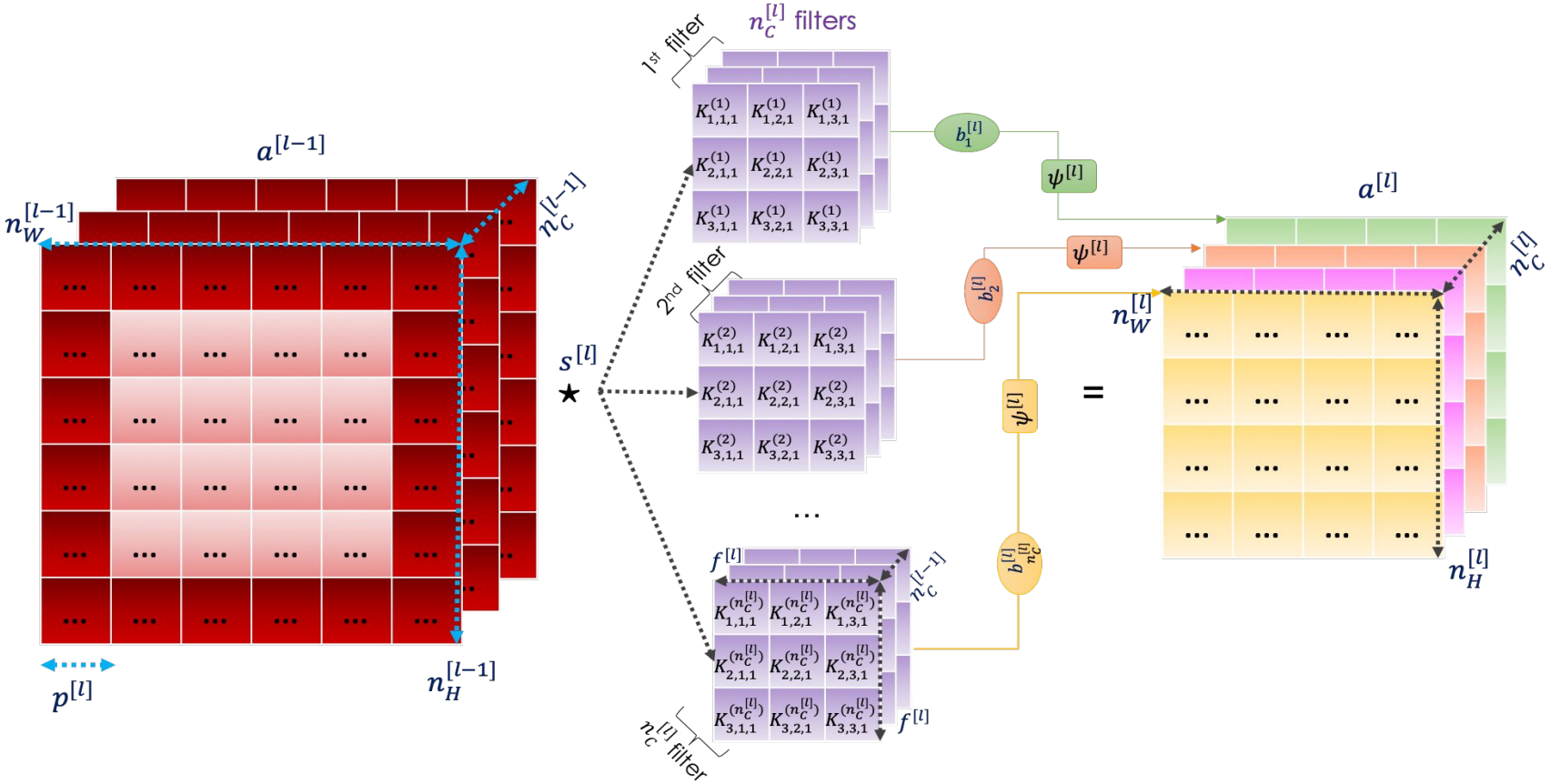
With:

$$n_{H/W}^{[l]} = \left\lfloor \frac{n_{H/W}^{[l-1]} + 2p^{[l]} - f^{[l]}}{s^{[l]}} + 1 \right\rfloor ; s > 0$$
$$= n_{H/W}^{[l-1]} + 2p^{[l]} - f^{[l]}; s = 0$$
$$n_C^{[l]} = number\ of\ filters$$

The `learned parameters` at the lth layer are:

- **Filters** with $(f^{[l]} \times f^{[l]} \times n_C^{[l-1]}) \times n_C^{[l]}$ parameters
- **Bias** with $(1 \times 1 \times 1) \times n_C^{[l]}$ parameters (broadcasting)

The below image shows an illustration of a Convolution layer

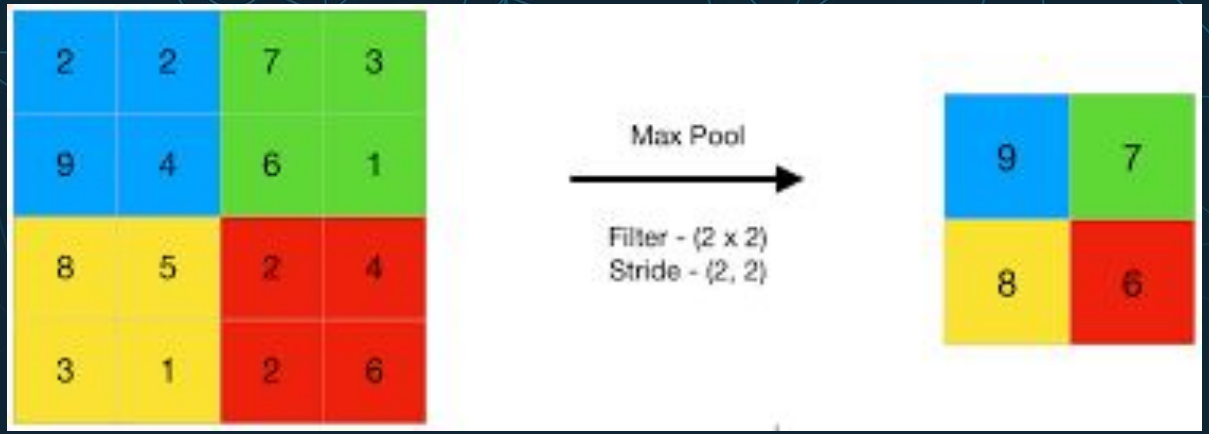


Pooling

The stride is the step taken in the convolutional product. A smaller stride allows to expand the size of the output and vice versa. Stride parameter is denoted by s.

Pooling is the step of summing up the information to downsample the image's features. The operation affects only the dimensions and is carried out through each channel so, it doesn't affect n_C.Given an image, we slide a filter, with no parameters to learn, following a certain stride, and we apply a function on the selected elements.In convention, we usually set *f*=2 and consider *s*=2.

In this project we used Max pooling, in which, maximum value is returned, among all the elements in the filter. The below image is an illustration of Max Pooling.



Notations and Mathematical equations used in pooling at l^{th} layer are shown in the below image

`Input` : $a^{[l-1]}$ with size $(n_H^{[l-1]}, n_W^{[l-1]}, n_C^{[l-1]})$, $a^{[0]}$ being the image in the input

`Padding` : $p^{[l]}$(rarely used), `stride` : $s^{[l]}$

`Size` of the pooling filter: $f^{[l]}$

`pooling function` : $\phi^{[l]}$

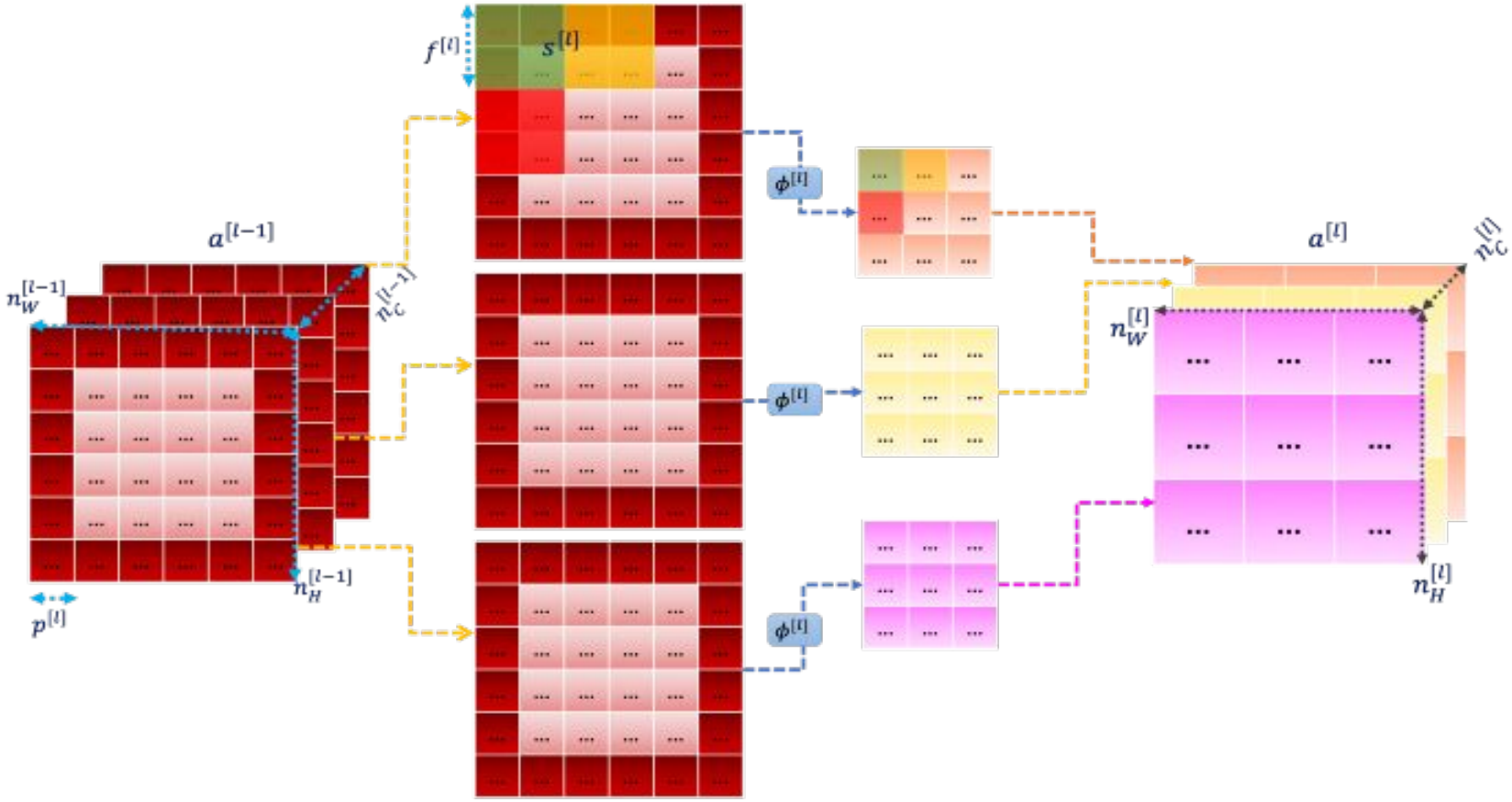
`Output` : $a^{[l]}$ with size $(n_H^{[l]}, n_W^{[l]}, n_C^{[l]} = n_C^{[l-1]})$

$$a_{x,y,z}^{[l]} = pool(a^{[l-1]})_{x,y,z} = \phi^{[l]}((a_{x+i-1,y+j-1,z}^{[l-1]})_{(i,j)\in[1,2,...,f^{[l]}]^2})$$
$$dim(a^{[l]}) = (n_H^{[l]}, n_W^{[l]}, n_C^{[l]})$$

With

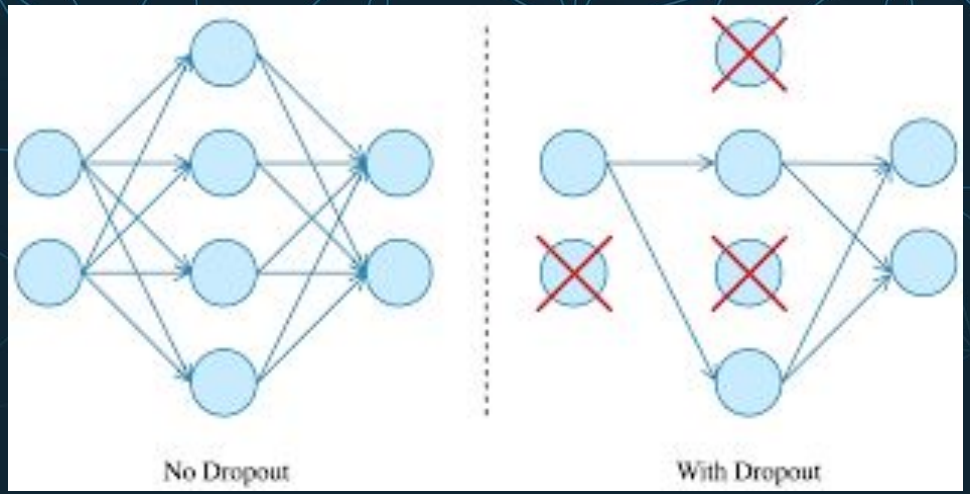
$$n_{H/W}^{[l]} = \left\lfloor \frac{n_{H/W}^{[l-1]} + 2p^{[l]} - f^{[l]}}{s^{[l]}} + 1 \right\rfloor ; s > 0$$
$$= n_{H/W}^{[l-1]} + 2p^{[l]} - f^{[l]}; s = 0$$
$$n_C^{[l]} = n_C^{[l-1]}$$

The below image shows an illustration of a Pooling Layer



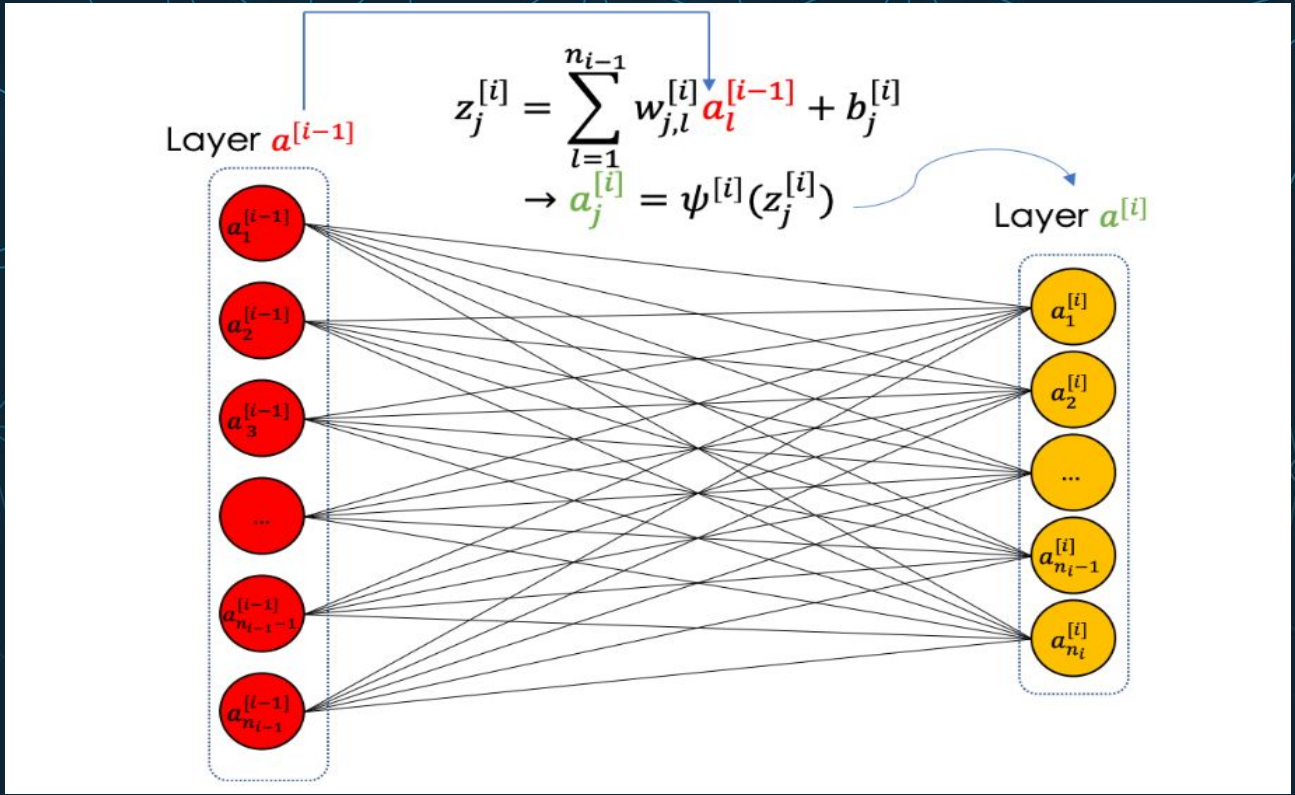
DropOut Layer

During training, some number of layer outputs are randomly ignored or "*dropped out*." Dropout is used to prevent a model from overfitting. Dropout works by randomly setting the outgoing edges of hidden units (neurons that make up hidden layers) to 0 at each update of the training phase.



Fully Connected Layer

A fully connected layer is a finite number of neurons which takes a vector as an input and returns another one. The below image is an illustration of a Fully Connected Layer



The below image shows the notations and mathematical equations used in a Fully Connected Layer

In general, considering the jth node of the ith layer we have the following equations:

$$z_j^{[i]} = \sum_{l=1}^{n_{i-1}} w_{j,l}^{[i]} a_l^{[i-1]} + b_j^{[i]}$$
$$\rightarrow a_j^{[i]} = \psi^{[i]}(z_j^{[i]})$$

The input $a^{[i-1]}$ might be the result of a convolution or a pooling layer with the dimensions $(n_H^{[i-1]}, n_W^{[i-1]}, n_C^{[i-1]})$.
In order to be able to plug it into the fully connected layer we `flatten` the tensor to a `1D vector` having the dimension: $(n_H^{[i-1]} \times n_W^{[i-1]} \times n_C^{[i-1]}, 1)$, thus:

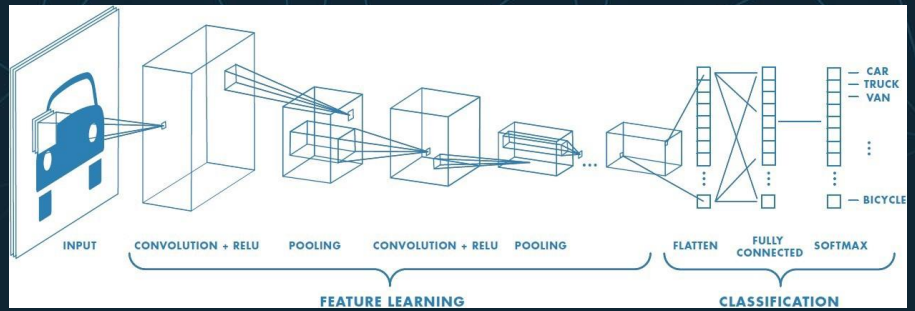
$$n_{i-1} = n_H^{[i-1]} \times n_W^{[i-1]} \times n_C^{[i-1]}$$

The `learned parameters` at the lth layer are:

- **Weights** $w_{j,l}$ with $n_{l-1} \times n_l$ parameters
- **Bias** with n_l parameters

Implementation

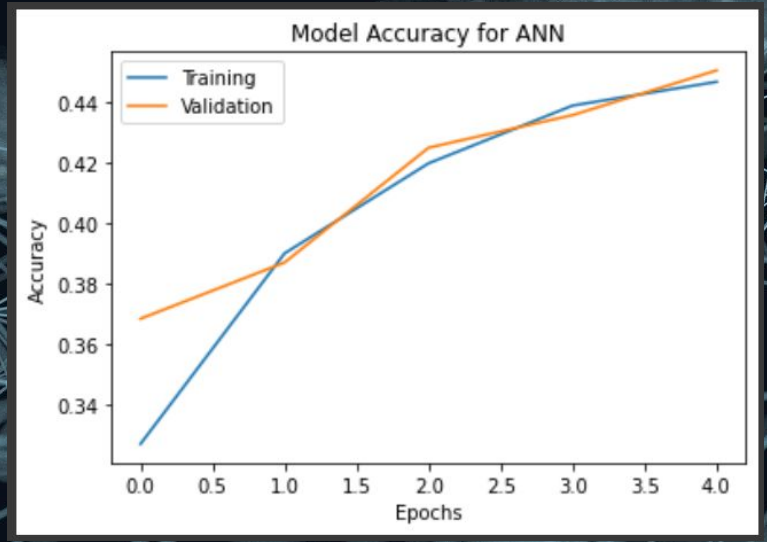
- Used Keras to build and train model
- Implemented traditional ANN first with 2 hidden, 1 input and 1 output layer. It consisted of 2500 nodes and used ReLU in hidden layers. Output layer had Sigmoid as its activation function.
- Implemented CNN next with Convolution, Max Pooling and Dropout layers with 32, 64,128 feature maps. It was followed by flattening of input and a fully connected network with 2 hidden layers of 128 and 256 nodes along with ReLU as its activation function.



Analysis & Results (1)

For ANN :-

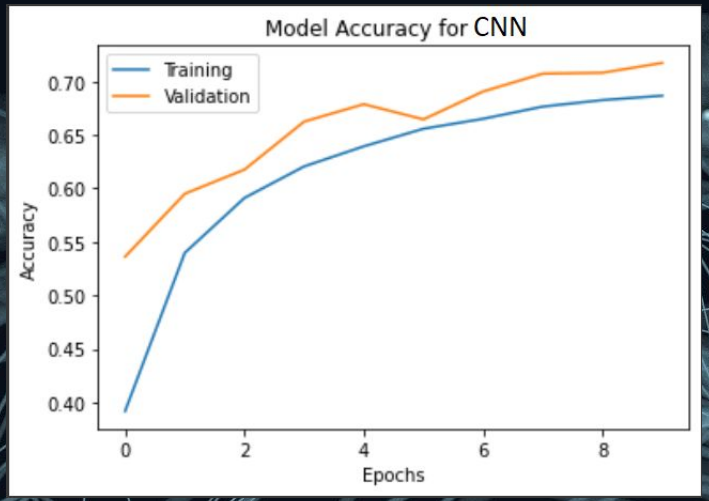
- We achieved **45.06%** at the end of 5 epochs
- The plot for accuracy on the training and validation set with the number of epochs is -

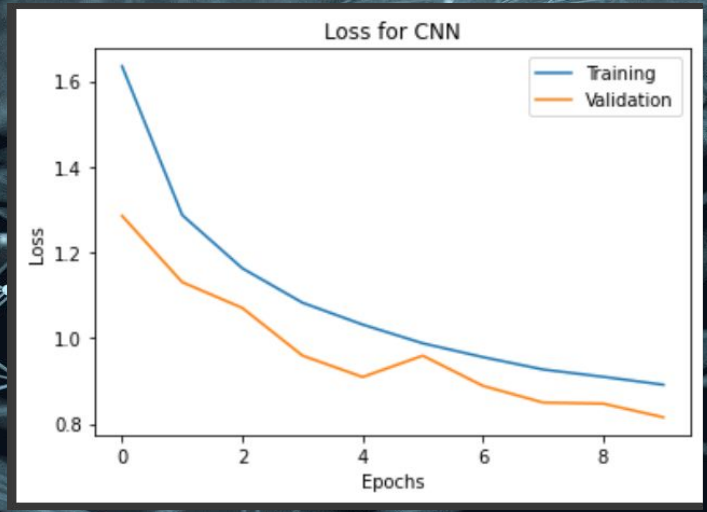


Analysis & Results (2)

For CNN :-

- We achieved **71.74%** accuracy at the end of 10 epochs
- The plot for Accuracy vs Epochs and Loss vs Epochs for CNN are as follows:- :-





Conclusion

Even at 5 epochs CNN outperformed ANN by almost 20% accuracy. Which brings us to the conclusion that **CNN is much better than ANN for object identification task** and we achieved an accuracy of **71.74%** using CNNs.

Contributions of each member

Satwik Murarka - Coded the algorithms, researched ideas, reviewed the ppt
Praveen Prakash - Prepared the presentation, reviewed the code, researched ideas
Mihir Nandawat - Prepared the presentation, reviewed the code, researched ideas

References-

- https://media.geeksforgeeks.org/wp-content/uploads/20190721025744/Screenshot-2019-07-21-at-2.57.13-AM.png
- https://towardsdatascience.com/convolutional-neural-networks-mathematics-1beb3e6447c0
- http://introtodeeplearning.com/slides/6S191_MIT_DeepLearning_L3.pdf
- https://machinelearningmastery.com/dropout-for-regularizing-deep-neural-networks/
- https://youtu.be/iaSUYvmCekI